

Math Primer

(13. Optimization III – Numerical Optimization)

Contents



- ▶ Gradient Descent
- ▶ Conditions for local maxima/minima
- ▶ Gradient Descent
- ▶ Newton's Method



JL Lagrange
(1736-1813)

Gradient Descent (Univariate)



► We can converge to minima iteratively by using a numerical marching scheme. The most popular algorithm is 'gradient descent'. The numerical methods become useful when the problems are complex and there is no analytical way to derive optimal solution. The process is as follows

- start with an initial guess x_0 .
- evaluate the gradient at that point, which will give the direction of descent.
- use a step size (learning rate) to descend along the gradient and move to a new point.
- continue until there is no significant improvement in the objective function.

$$x_{k+1} = x_k - \eta \left(\frac{dy}{dx} \right)$$

↓
Learning rate

Gradient Descent (Univariate)



- Let's take an example. Suppose we want to find the minima of $y = x^2 - 2x$. The first order condition yields $2x - 2 = 0 \Rightarrow x = 1$. Second order condition yields $2 > 0$. So, we have the minima at $x = 1$ and the optimal value is $y = -1$

Now we employ the gradient descent algorithm

$$x_{k+1} = x_k - \eta \left(\frac{dy}{dx} \right)$$

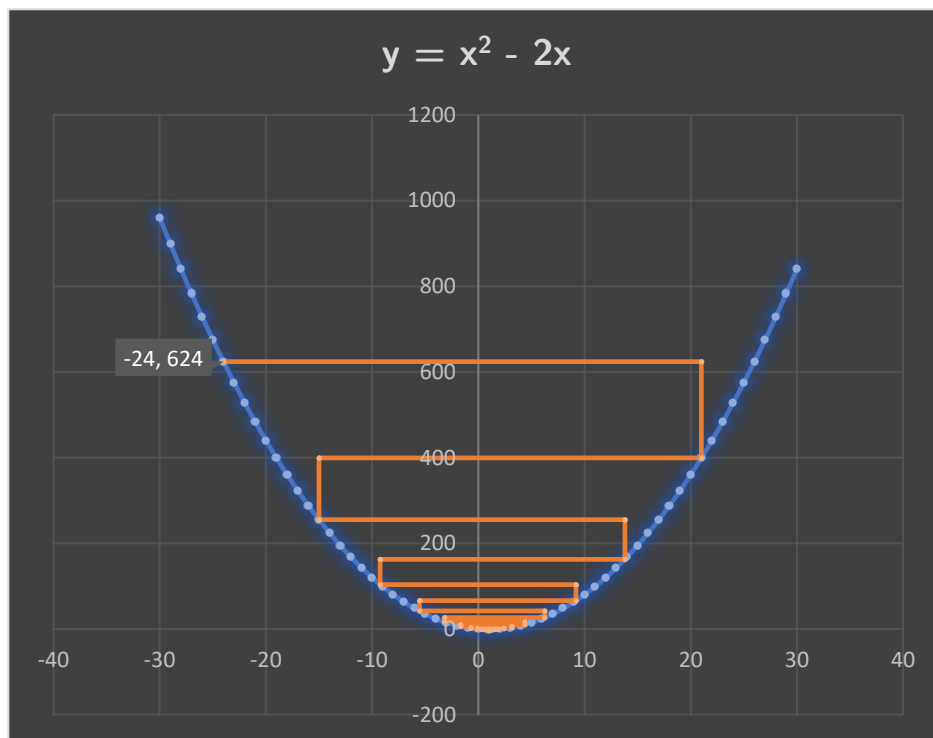
In our case, it becomes

$$x_{k+1} = x_k - \eta(2x_k - 2)$$

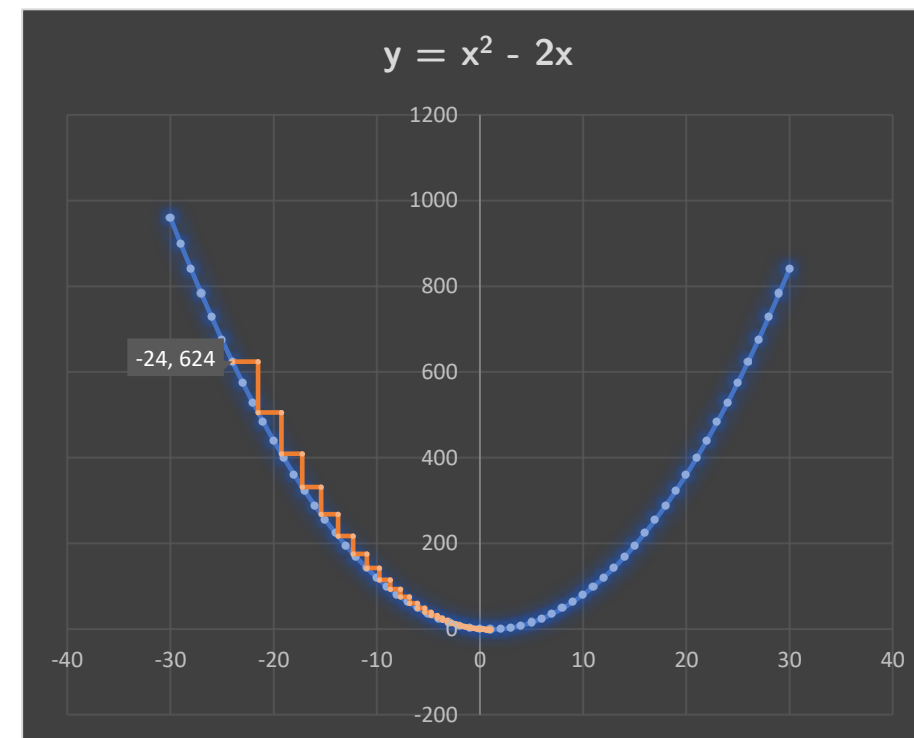
Gradient Descent (Univariate)

- ▶ With low values of learning rate (η) the solution will converge slowly
- ▶ With high values of learning rate (η) the solution will converge faster, but has the risk of overstepping the minima

Learning rate = 0.9



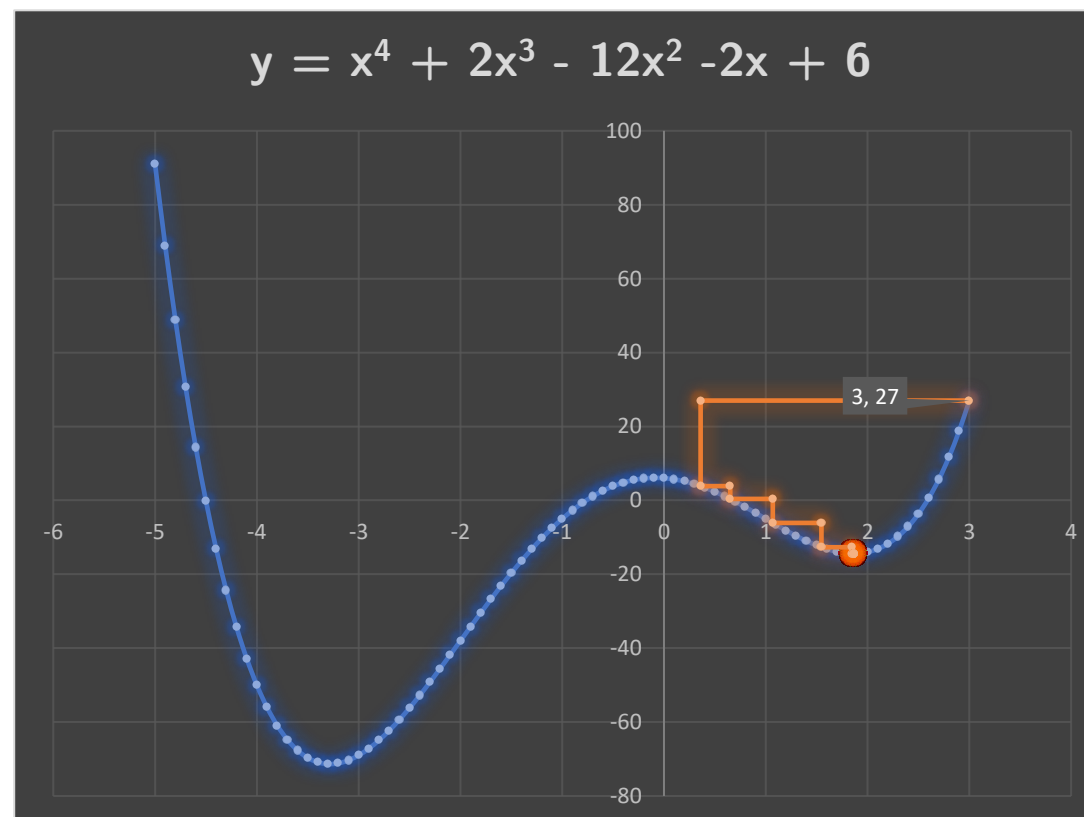
Learning rate = 0.05



Gradient Descent (Univariate)



- Sometimes if the objective function has multiple local minima (non-convex), there is a chance that the algorithm gets stuck in a local minimum instead of the global minimum. That's why we typically work with convex objective functions which have a single minimum.



Gradient Descent (Multivariate)



- ▶ We can extend the formula to optimize a multivariable function. Instead of a single derivative, use gradient vector. Everything else remains same overall !

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla \mathbf{z}$$

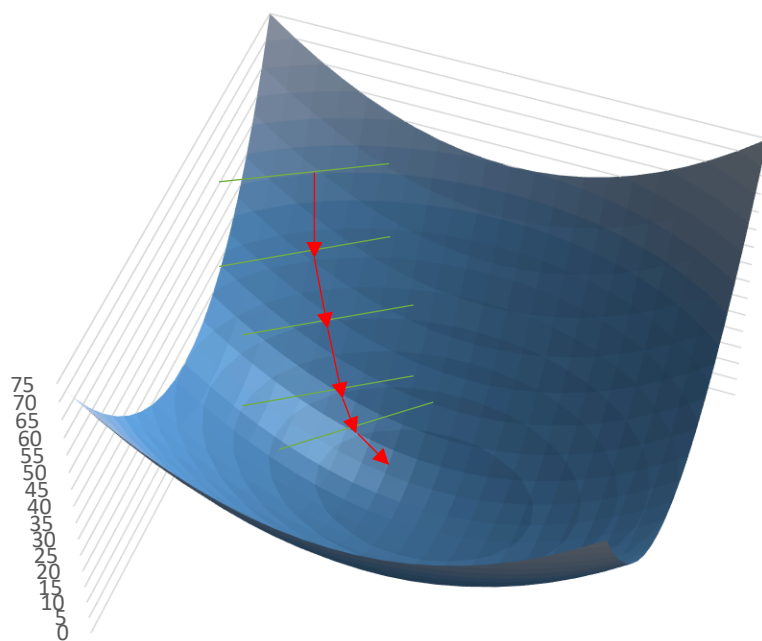
For example, in 2D, we can formulate the gradient descent algorithm as

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \eta \begin{pmatrix} f_x \\ f_y \end{pmatrix}$$

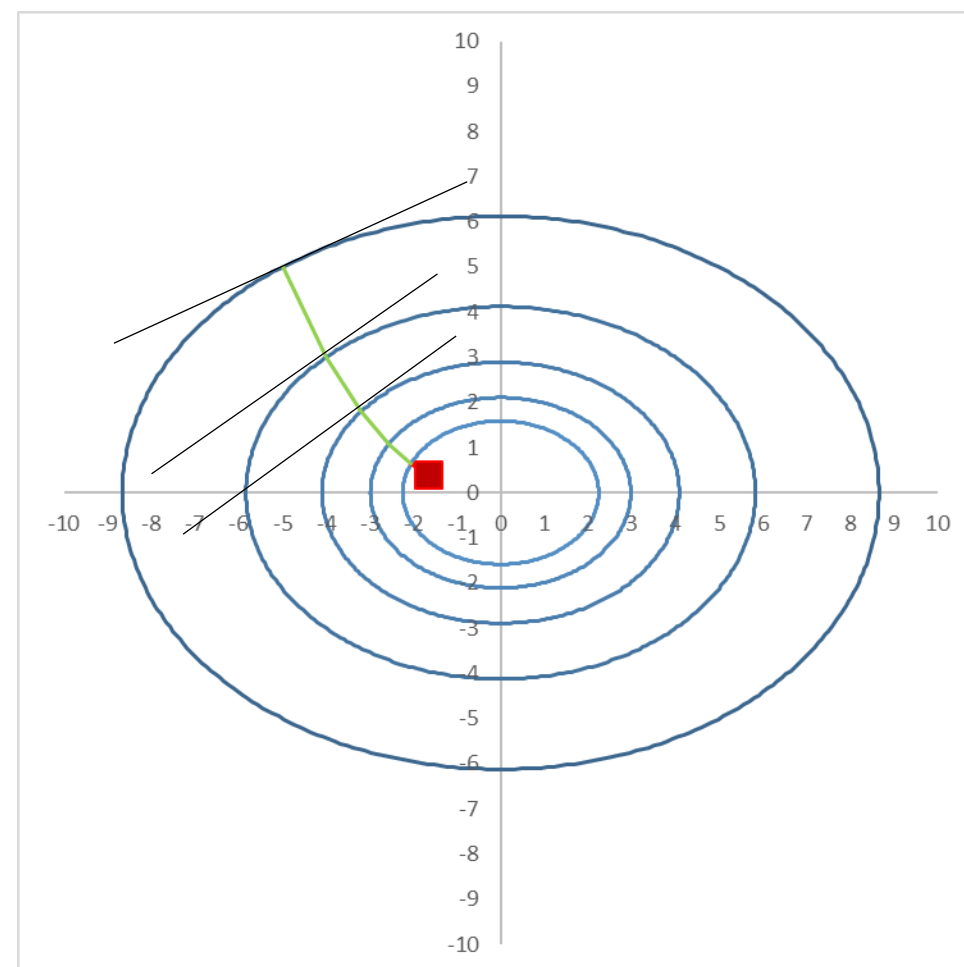
Gradient Descent in 2D (Example)

► As an example, consider the following problem

$$\min z = f(x, y) = x^2 + 2y^2$$

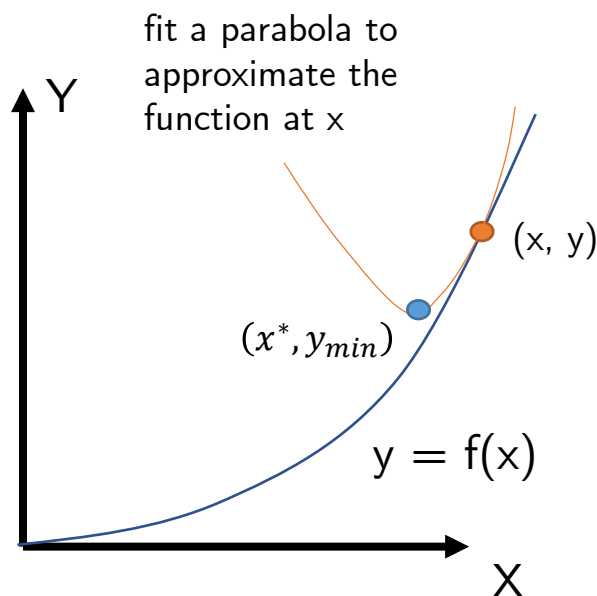


$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \eta \begin{pmatrix} 2x \\ 4y \end{pmatrix}$$



Newton's Method (Univariate)

- ▶ Another popular numerical method is Newton's algorithm. This can be viewed as using an adaptive learning rate as opposed to a constant one



Fitted parabola :

$$f(x + t) = f(x) + tf'(x) + \frac{t^2}{2} f''(x) \quad (\text{Taylor Series})$$

One can move along the parabola by changing t
The minima of the parabola is achieved by setting

$$\frac{\partial f(x + t)}{\partial t} = 0 \quad t = -\frac{f'(x)}{f''(x)}$$

The minima is achieved at $x^* = x + t = x - \frac{f'(x)}{f''(x)}$

If we compare with GD,
Newton's method uses adaptive learning rate $\eta = \frac{1}{f''(x)}$

Newton's Method (Multivariate)



- ▶ We can easily extend Newton's method to multivariate settings. For multivariate problem, the 1st derivative is replaced by gradient and the 2nd derivative is replaced by Hessian matrix

$$\mathbf{x} \rightarrow \mathbf{x} - \mathbf{H}^{-1} \nabla f$$

Homework – Write the equation to iterate under Newton's method to solve for

$$\min f(x, y) = x^4 + (y - 2)^4$$